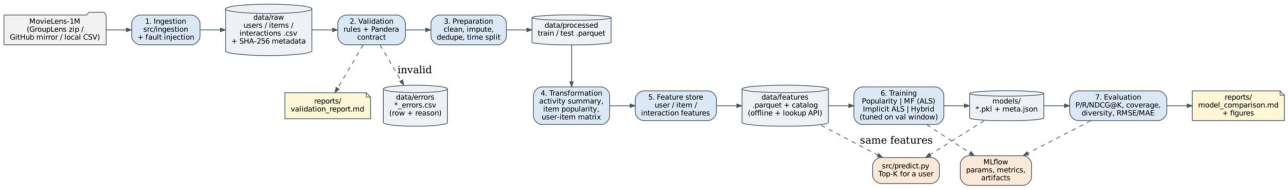
End-to-End Data Management Pipeline for a Recommendation System

DM4ML – Assignment 2 | Name: Abhinand NTK | ID: 2025ag05637 | Repo: https://github.com/Abhinand-NTK/Recommendation-pipeline

1. Problem and dataset

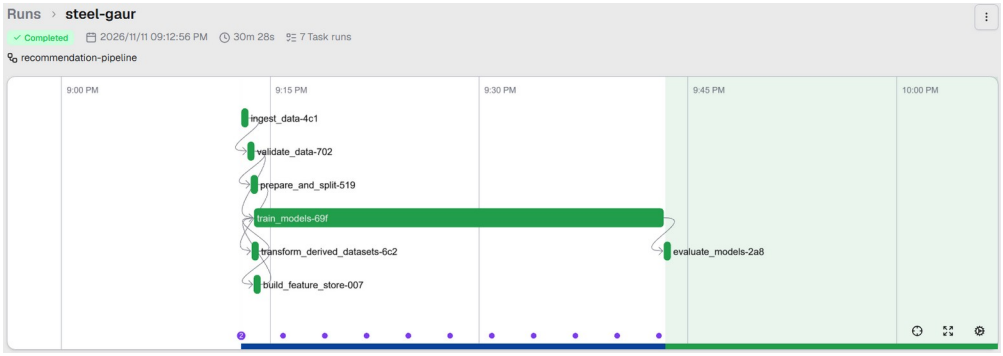
The goal is to recommend movies a user is likely to enjoy, using past ratings and item/user metadata. The pipeline uses the public **MovieLens-1M** dataset (6,040 users, 3,883 movies, 1,000,209 ratings on a 1–5 scale, 2000–2003), which provides all three required entities: user details (gender, age band, occupation, zip code), item details (title, year, genres) and timestamped user–item interactions. Because the real data is already clean, ingestion appends a documented, seeded set of 2,475 corrupt records (missing/invalid ratings, bad timestamps, duplicates, orphan IDs) so the validation layer is exercised on realistic failures; this can be switched off in the config.

2. Architecture and design decisions



Each stage is an independent module with a run(cfg) entry point that reads its inputs from disk and writes versioned outputs, so stages can be re-run in isolation and orchestrated as a DAG. All parameters live in one YAML config. Key decisions:

- **Layered storage** – raw (CSV, kept as text so type errors survive) → validated → processed (Parquet) → features → models, plus a separate error layer. Raw files are SHA-256 hashed.
- **Validation** – 29 rules across three tables. ERROR-severity rows are moved to data/errors/ with a machine-readable reason, WARNING rows (e.g. missing gender/genre) are kept and imputed later. Referential integrity is checked against the validated user/item tables, and a Pandera schema contract guards the clean output. Result: users 99.51%, items 99.49%, interactions 99.76% pass; 2,400 interactions rejected.
- **Time-based split** – a global cut-off at the 80th time percentile (2000-12-02): 800,164 train vs 200,045 test rows. This mimics production (train on the past, predict the future) and prevents leakage; a per-user chronological split is available as an option.
- **Feature store** – a custom, file-based layer: 34 user, 31 item and 7 interaction features computed as-of the cut-off from train data only. A Python registry is the single source of truth and generates the feature catalog. The FeatureStore API (with defaults for cold users) is used by the hybrid model during training, by tuning, and by predict.py at serving time.
- **Tuning without test leakage** – hybrid weights are grid-searched on the last 10% of the train period with features rebuilt for that window; best: genre 0.1, popularity 3.0. Every grid point is a nested MLflow run.
- **Orchestration** – a Prefect 3 flow with explicit task dependencies, retries on ingestion and an optional daily schedule; plus a dependency-aware plain-Python runner (--from-stage, --only).



Successful Prefect run (steel-gaur): all 7 tasks Completed in dependency order, 30 m 28 s.

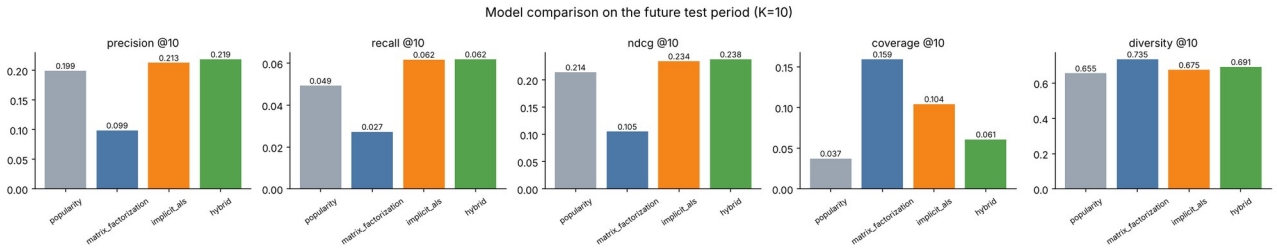
3. Models

Model	Description
Popularity (baseline)	Ranks items by number of ratings; Bayesian-average rating as rating prediction
Matrix factorization	Explicit biased MF, r = μ + b_u + b_i + p_u·q_i, 64 factors, ALS, 15 iterations
Implicit ALS	Weighted MF on implicit 'watched' feedback (Hu et al., 2008), 128 factors, λ=100
Hybrid	z-score blend of implicit-ALS score + user–item genre affinity (feature store) + popularity prior

4. Results

Evaluated on 1,122 warm-start test users (relevant = rating ≥ 4; candidates exclude items already seen in training).

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
popularity	0.1995	0.0493	0.2141	0.0374	0.6550	0.9608	0.7611
matrix_factorization	0.0989	0.0273	0.1052	0.1592	0.7346	0.8939	0.6958
implicit_als	0.2129	0.0617	0.2344	0.1043	0.6745	n/a	n/a
hybrid	0.2189	0.0619	0.2380	0.0609	0.6913	n/a	n/a



Findings. (1) Under a strict future split, popularity is a strong baseline: the test period is dominated by heavily-rated titles. (2) Explicit MF gives the best rating accuracy (RMSE 0.894 vs 0.961) but ranks poorly, because predicting a 5-star rating for a niche film is different from predicting what a user will watch next. (3) Implicit ALS, which models what users choose, beats popularity on every accuracy metric while covering ~3× more of the catalogue. (4) The hybrid gives the best precision, recall and NDCG, at the cost of coverage – a classic accuracy vs. coverage trade-off that the weights control. The genre signal adds little on top of collaborative filtering for warm users.

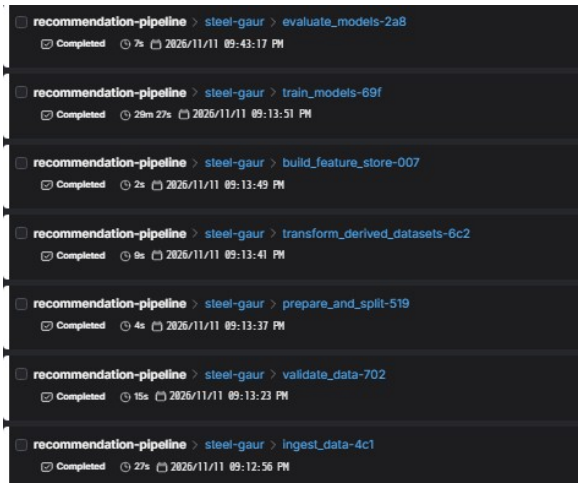
5. Limitations and future work

- Only warm users are ranked; cold-start users (in test but not train) would need the content/demographic features as a fallback model.
- Hybrid tuning favours popularity, which lowers catalogue coverage; a diversity or coverage term in the objective (or re-ranking) would balance this.
- MovieLens-1M is dated and dense; fault injection simulates, but does not fully reproduce, real production data issues. The feature store is file-based (no online low-latency serving); Feast or a DB-backed store would be the next step.
- Hyper-parameters for the MF models were set by hand; a broader search (e.g. Optuna) on the validation window could improve results.

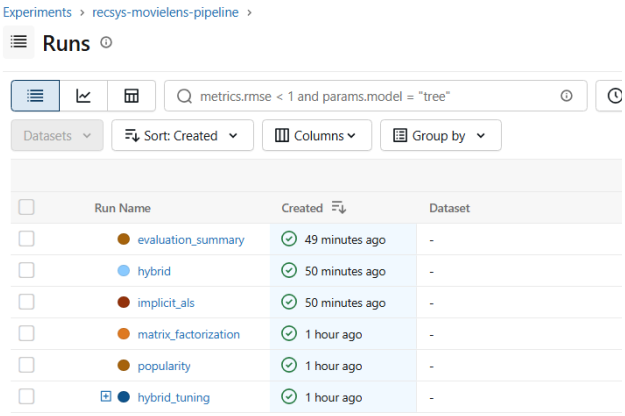
6. Tools

Python 3.13, Pandas, NumPy/SciPy (ALS implementations), Pandera (schema contracts), PyArrow/Parquet (storage), MLflow (tracking: params, metrics, artifacts, nested tuning runs), Prefect 3 (orchestration), Matplotlib & Graphviz (figures), pytest (unit tests).

Appendix – Evidence of execution



A1 – Prefect task runs: all 7 tasks Completed



A2 – MLflow: one tracked run per model + nested tuning runs

```
# Data Validation Report

Generated: 2026-11-11T17:35:53.932951+00:00

## Summary

| Table | Raw rows | Valid rows | Rejected rows | Pass rate |
|---|---|---|---|---|
| users | 6,085 | 6,055 | 30 | 99.5070% |
| items | 3,913 | 3,893 | 20 | 99.4889% |
| interactions | 1,002,609 | 1,000,209 | 2,400 | 99.7606% |

## Rule results

| Table | Rule | Severity | Failed rows | Description |
|---|---|---|---|---|
| users | `missing_user_id` | ERROR | 0 | user_id is null |
| users | `user_id_not_integer` | ERROR | 0 | user_id must be an integer |
| users | `duplicate_user_id` | ERROR | 15 | user_id appears more than once (first occurrence kept) |
| users | `invalid_age` | ERROR | 15 | age must be one of the MovieLens age buckets [1, 18, 25, 35,
| users | `invalid_gender_value` | ERROR | 0 | gender must be M or F |
| users | `invalid_occupation` | ERROR | 0 | occupation code must be in [0, 20] |
| users | `missing_gender` | WARNING | 15 | gender missing - imputed as 'unknown' during preparation
| users | `missing_age` | WARNING | 0 | age missing - imputed as 'unknown' during preparation |
| users | `missing_occupation` | WARNING | 0 | occupation missing - imputed as 'unknown' during prep
| users | `missing_zip_code` | WARNING | 0 | zip_code missing - imputed as 'unknown' during preparat
| items | `missing_item_id` | ERROR | 0 | item_id is null |
```

A3 – Validation report (reports/validation_report.md)

	user_id	item_id	rating	timestamp	error_reason
2	900102	3638	4	9.7E+08	orphan_user_id
3	4771	1914		9.63E+08	missing_rating
4	1752	965		9.76E+08	missing_rating
5	38	1835	6	9.78E+08	rating_out_of_range
6	900277	162	5	9.74E+08	orphan_user_id
7	3567	1259	4.x	9.67E+08	rating_wrong_type
8	1038	34		9.75E+08	missing_rating
9	329	3528	3	abc	timestamp_not_numeric
10	5621	2412		9.59E+08	missing_rating
11	3841	3103	3	-5	timestamp_out_of_range
12	3054	1947		9.71E+08	missing_rating
13	1958	800040	3	9.94E+08	orphan_item_id
14	900190	3152	3	9.57E+08	orphan_user_id
15	1284	3752	five	9.83E+08	rating_wrong_type
16	4086	2142		9.65E+08	missing_rating
17	4186	3578		1.02E+09	missing_rating
18	2179	3044	4	2000-13-4	timestamp_not_numeric
19		3246	4	9.6E+08	missing_user_id
20	1207	50	five	9.75E+08	rating_wrong_type

A4 – Error dataset with error_reason (data/errors/)

```
# Feature Catalog

Generated automatically from `src/features/registry.py`. All features are computed from the training split only, as-of the split cut-off, to avoid leakage. Access them through `src.features.feature_store.FeatureStore`.

Null % on user features = users with no activity before the cut-off (cold users); the FeatureStore fills them with documented defaults (global mean, zero counts).

## User features (16)

| Feature | Type | Description | Source | Logic | Used by | Null % | Example |
|---|---|---|---|---|---|---|---|
| `user_activity_count` | int | Number of ratings the user made before the cut-off | train | count(ratings) per user | hybrid, eval segments | 0.00% | 0 |
| `user_distinct_items` | int | Distinct items rated | train | nunique(item_id) per user | analysis | 10.82% | 31.0 |
| `user_avg_rating` | float | Mean rating given by the user | train | mean(rating) per user | hybrid, MF bias init | 10.82% | 4.323 |
| `user_rating_std` | float | Std-dev of the user's ratings | train | std(rating) per user, 0 if single rating | analysis | 10.82% | 0.599 |
| `user_rating_bias` | float | How much more generous than average the user is | train | user_avg_rating - global_mean_rating | analysis | 10.82% | 0.732 |
| `user_share_liked` | float | Share of ratings >= relevance threshold | train | mean(rating >= 4) | analysis | 10.82% | 0.935 |
| `user_preferred_genre` | category | Genre the user rates most often | train + items | argmax of user genre profile | hybrid, cold-start fallback | 10.82% | drama |
| `user_preferred_genre_share` | float | Share of ratings in the preferred genre | train + items | max of user genre profile | analysis | 10.82% | 0.429 |
| `user_genre_entropy` | float | Breadth of taste (Shannon entropy of genre profile) | train + items | -sum(p*log p) over genre shares | analysis | 10.82% | 1.886 |
| `user_days_since_last_interaction` | float | Recency of the user's last rating | train | (as_of - max(event_time)) in days | analysis, segmentation | 10.82% | 0.0 |
| `user_tenure_days` | float | Days between first and last rating | train | max(event_time) - min(event_time) in days | analysis | 10.82% | 0.01 |
| `user_active_days` | int | Number of distinct days with activity | train | nunique(date(event_time)) | analysis | 10.82% | 1.0 |
| `user_gender` | category | Gender (M/F/U) | users | standardised, U if missing | analysis | 0.00% | F |
| `user_age_group` | category | MovieLens age bucket label | users | decoded from age code | analysis | 0.00% | Under 18 |
| `user_occupation` | category | Occupation name | users | decoded from occupation code | analysis | 0.00% | K-12 student |
| `genre_profile_{g}` (19 cols) | float | Share of the user's ratings that fall in genre {g} | train + items | count per genre / total genre tags | hybrid (genre affinity) | 10.82% | 0.036 |
```

A5 – Feature catalog generated from the registry (reports/feature_catalog.md)